DeepRL – Discrete action space

from zero to hero

What is policy gradient? a recap

Long story short, this is the elephant in the room:

$$\nabla J(\theta) \approx \mathbb{E}_\pi \left[G_t \frac{\nabla \pi(a|s)}{\pi(a|s)} \right]$$

In poor words, we select an action, and we "reinforce it" proportional to the return of it: assuming all $\mathbf{G_t > 0}$, then all actions will be reinforced, but the one with the highest return will prevail, pushing the other ones closer to 0.

Can’t we just use a table

Table dimensionality gets to |S|x|A|, however, policy gradient still works:

$$\pi(a|s,\theta) = \frac{e^{h_\theta(s,a)}}{\sum_{a' \in A} e^{h_\theta(s,a')}}$$

nothing stops us from representing the **h(s,a)** as just entries is a table, and use Policy gradient to learn them

However, states usually are similar in some ways, and we can use this property.

This will allow us to “fill the table” without actually having to sample multiple samples per entry, but use generalization to extrapolate that information on the fly (similar states will have similar consequences, if the similarity is learnt in the “consequence” space)

Maybe linear is the way

You then have seen that we can parametrize the state in different ways (tile coding for example), and assume there is some sort of linear correlation between the policy and the state

$$\pi(a|s,\theta) = \frac{e^{h_\theta(s,a)}}{\sum_{a' \in A} e^{h_\theta(s,a')}}$$

$$h_\theta(s,a) = \theta^T x(s,a)$$

However, nobody tells you how to build **s**, and for this to work you need to build an **s** such that being close in the state representation space, you are also close in the "consequence space" (think about chess, very similar boards might be completely different, but very different boards, might at the end be pretty much equivalent, if the differences are irrelevant)

Maybe linear is the way... or maybe not

So, the whole point is to learn a good representation of **s**... and we just saw how neural networks are amazing generalization machine (spoiler: you can consider the neural network as a great preprocessing pipeline, and the head of it being just a linear regression... however the pre-processing this way is also optimized to improve performances, and being learnt, implies having close to no limit)

$$\pi(a|s,\theta) = \frac{e^{h_\theta(s,a)}}{\sum_{a' \in A} e^{h_\theta(s,a')}} \qquad h_\theta(s,a) = f(s,a;\theta)$$

Long life to non-convex black box function approximators

Given that, now we are left with the non-trivial problem to actually make this 2 worlds work together.

However, it's easier to be said than to be done, as there is a very big friction between the two approaches (RL and DL)... which one?

Long life to non-convex black box function approximators

Given that, now we are left with the non-trivial problem to actually make this 2 worlds work together.

However, it's easier to be said than to be done, as there is a very big friction between the two approaches (RL and DL)... which one?

I.I.D.

Long life to non-convex black box function approximators

Given that, now we are left with the non-trivial problem to actually make this 2 worlds work together.

However, it's easier to be said than to be done, as there is a very big friction between the two approaches (RL and DL)... which one?

I.I.D.

Indeed, RL until now didn't have to make any big assumption on the distribution of the data, as tabular cases are immune to correlation between samples... well, not SGD (and thus not neural networks)
... reason why you saw Experience Replay being used in DQN

Long life to non-convex black box function approximators

However, we cannot use them in policy gradient, as that would imply using off-policy data, which is not really the best... for another very painful reason, can you guess it?

VARIANCE

(aka importance sampling coeff)

Of course variance is a problem, because now "entries on the table" are instead representation in the network... if by chance you get one ratio giant, it will overshadow the gradient from the other samples... screwing everything up

So can’t we just use the policy gradient we studied?

Yes and no. You have to be very careful with the two problems just stated.
The rule of thumb is: if there is a way to do the same with less variance, then do it that way

How can we introduce variance?

1. Reward function: indeed $\mathbf{G_t}$ might have high variance, and when you do $\mathbf{G_t \nabla \ ln(\pi(a|s))}$, the gradient for that sample is weighted according to that scalar… if you have 100 samples in your minibatch, and one has a return 100x bigger, the gradient of that sample will overshadow the gradient produced by the others

Define the reward function accurately

So can’t we just use the policy gradient we studied?

Yes and no. You have to be very careful with the two problems just stated.
The rule of thumb is: if there is a way to do the same with less variance, then do it that way

How can we introduce variance?

1. Reward function
2. Stochastic optimization: at the end of the day, NN are just differentiable models, and being non-convex, means to having to deal with a lot of bad scenarios… in addition to poor conditioning, you might have cliffs, exploding gradients, flat regions, ill-conditioning

Use advanced optimizers that use adaptive steps and momentum

So can’t we just use the policy gradient we studied?

Yes and no. You have to be very careful with the two problems just stated.
The rule of thumb is: if there is a way to do the same with less variance, then do it that way

How can we introduce variance?

1. Reward function
2. Stochastic optimization
3. Importance Sampling ratio: if using off-policy data is nice, but the importance sampling ratio might explode if the denominator is much smaller than the numerator

Pay attention to the algorithm that you use, if it handles this case or not

Is there hope to fix them?

Solutions:

1. Reward function: redefine the reward function to be well behaved, normalize the returns, or use more advanced approaches proposed in literature for DeepRL (PopArt, Value Normalization)
2. Stochastic optimization: the family of momentum based optimizers and adaptive gradient optimizers tries to solve this problem since 20 years, Adam is always your best friend (of if you really cannot converge, maaaybe consider K-fac, if your library has it available)
3. Importance Sampling ratio: can we do anything about it?

Why is Actor critic not enough

The title is kind of a lie, Actor critic, moreover Advantage Actor critic (aka A2C), is still a very good algorithm for DeepRL, however, the fact that we have to discard the data after one step of learning, is kind of bummer.

Can we do better?

Why is Actor critic not enough

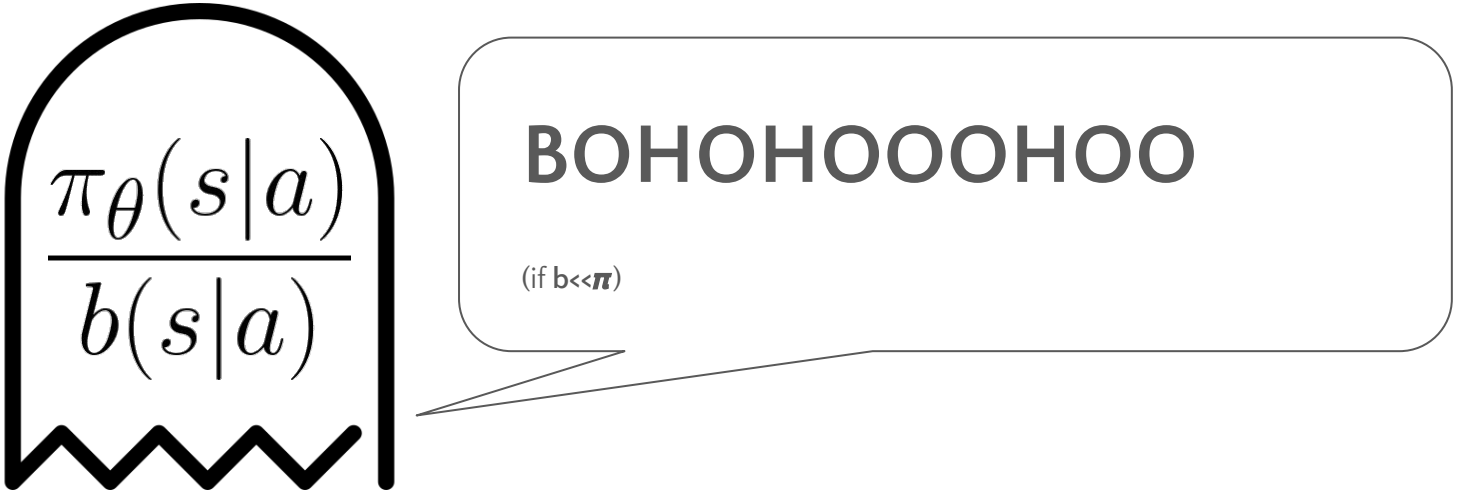
The title is kind of a lie, Actor critic, moreover Advantage Actor critic (aka A2C), is still a very good algorithm for DeepRL, however, the fact that we have to discard the data after one step of learning, is kind of bummer.

Can we do better? Of course, why not reusing it!... or not?

Why is Actor critic not enough

The title is kind of a lie, Actor critic, moreover Advantage Actor critic (aka A2C), is still a very good algorithm for DeepRL, however, the fact that we have to discard the data after one step of learning, is kind of bummer.

Can we do better? Of course, why not reusing it!... or not?



Tackling importance sampling weights

How can reuse the data?

1. Sample data from the environment using $\boldsymbol{\pi}$
2. update Value function using TD learning

$$L(\phi) = \sum (V_\phi(s) - sg[r + (1-d)\gamma V_\phi(s'))])^2$$

3. update Policy function using A2C

$$\begin{aligned} L(\theta) &= \sum (V_\phi(s) - (r + (1-d)\gamma V_\phi(s')))) \nabla \ln(\pi_\theta(a|s)) \\ &= \sum A_t \nabla \ln(\pi_\theta(a|s)) \end{aligned}$$

Tackling importance sampling weights

1. Sample data from the environment using $\boldsymbol{\pi}$
2. update Value function using TD learning

$$L(\phi) = \sum (V_\phi(s) - sg[r + (1-d)\gamma V_\phi(s'))])^2$$

3. update Policy function using A2C

here we need to use the data multiple times, however, after the first update, the policy will not be anymore the same

$$\begin{aligned} L(\theta) &= \sum (V_\phi(s) - (r + (1-d)\gamma V_\phi(s')))) \nabla \ln(\pi_\theta(a|s)) \\ &= \sum A_t \nabla \ln(\pi_\theta(a|s)) \end{aligned}$$

Tackling importance sampling weights

1. Sample data from the environment using $\boldsymbol{\pi}$
2. update Value function using TD learning

$$L(\phi) = \sum (V_\phi(s) - sg[r + (1-d)\gamma V_\phi(s'))])^2$$

3. update Policy function using A2C for N steps, corrected with IS

$$\begin{aligned} L(\theta) &= \sum \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A_t \nabla \ln(\pi_\theta(a|s)) \\ &= \sum \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A_t \frac{\nabla \pi_\theta(a|s)}{\pi_\theta(a|s)} = \sum \frac{A_t}{\pi_{\theta_{\text{old}}}(a|s)} \nabla \pi_\theta(a|s) \end{aligned}$$

Tackling importance sampling weights

1. Sample data from the environment using $\boldsymbol{\pi}$
2. update Value function using TD learning

$$L(\phi) = \sum (V_\phi(s) - sg[r + (1-d)\gamma V_\phi(s'))])^2$$

3. update Policy function using A2C for N steps corrected with IS

$$L(\theta) = \sum \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A_t \nabla \ln(\pi_\theta(a|s))$$

$$= \sum \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A_t \frac{\nabla \pi_\theta(a|s)}{\pi_\theta(a|s)} = \sum \frac{A_t}{\pi_{\theta_{\text{old}}}(a|s)} \nabla \pi_\theta(a|s)$$

We didn't solve it, we just hided it in the simplification... if we set a N too large, we still have the variance

TRPO

TRPO was introduced to tackle this problem. Instead of just optimizing that loss, it wants to constrain the problem in a safe area of search (in the parameter space), thus solving the same problem:

$$\underset{s.t.||\theta-\theta_{\text{old}}||_2^2 \leq \epsilon}{minimize} \sum A_t \frac{\nabla \pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$$

The problem is that staying close in the parameter space, doesn't mean to stay close in the policy space

TRPO

TRPO was introduced to tackle this problem. Instead of just optimizing that loss, it wants to constrain the problem in a safe area of search (in the parameter space), thus solving the same problem:

$$\underset{s.t. D_{KL}(\pi_\theta || \pi_{\theta_{\text{old}}}) \leq \epsilon}{minimize} \sum A_t \frac{\nabla \pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$$

Ideally we want this or some other distance measure in the distribution space, however, it's definitely not clear how to enforce such constraint

TRPO

Since estimating that KL term is too complicated, they first approximate it heuristically:

$$D_{KL}(\pi_\theta||\pi_{\theta'}) = \mathbb{E}_{\mu_{\pi_\theta}(s)}[D_{KL}(\pi_\theta(a|s)||\pi_{\theta'}(a|s))]$$

At this point, we further need to simplify it, considering an approximation:

$$\mathbb{E}_{\mu_{\pi_\theta}(s)}[D_{KL}(\pi_\theta(a|s)||\pi_{\theta'}(a|s))] \approx (\theta - \theta')^T F(\theta - \theta')$$

With **F** being the Fisher information matrix $F = \mathbb{E}_\pi[\nabla_\theta \log \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)^T]$

TRPO

We start with a quadratic form in weight space, and now we have a quadratic form in policy space:

$$||\theta - \theta'||^2 = (\theta - \theta')I(\theta - \theta')^T \implies \mathbb{E}_{\mu_{\pi_\theta}(s)}[D_{KL}(\pi_\theta(a|s)||\pi_{\theta'}(a|s))] \approx (\theta - \theta')^T F(\theta - \theta')$$

And the solution is given by the following equation (with some caution for the stepsize to stay in the trust region):

$$\theta' = \theta + \alpha F^{-1} \nabla_\theta J(\theta)$$

This is also done by Natural Gradient, but TRPO uses conjugate gradient to calculate the inverse, and in the meantime also obtaining the stepsize

extremely evident in gaussian policy

Algorithm 1 Trust Region Policy Optimization

Input: initial policy parameters θ_0, initial value function parameters ϕ_0

Hyperparameters: KL-divergence limit δ, backtracking coefficient α, maximum number of backtracking steps K

for $k = 0, 1, 2, ...$ **do**

Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.

Compute rewards-to-go $\hat{R}_t$.

Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.

Estimate policy gradient as

$$\hat{g}_k = \frac{1}{|\mathcal{D}_k|} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)|_{\theta_k} \hat{A}_t.$$

Use the conjugate gradient algorithm to compute

$$\hat{x}_k \approx \hat{H}_k^{-1} \hat{g}_k,$$

where $\hat{H}_k$ is the Hessian of the sample average KL-divergence.

Update the policy by backtracking line search with

$$\theta_{k+1} = \theta_k + \alpha^j \sqrt{\frac{2\delta}{\hat{x}_k^T \hat{H}_k \hat{x}_k}} \hat{x}_k,$$

where $j \in \{0, 1, 2, ...K\}$ is the smallest value which improves the sample loss and satisfies the sample KL-divergence constraint.

Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_\phi \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.

end for

TRPO relaxed

One way of solving constrained optimization (at least with equalities) is to use Lagrange multipliers, and indeed we can do the same here:

$$L(\theta) = \sum A_t \frac{\nabla \pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} - \lambda D_{KL}(\pi_\theta || \pi_{\theta_{\text{old}}})$$

However, **λ** it's quite tricky to pick... too low and you have variance problem, too high and you don't learn at all. What we can do is to use dual gradient descent:

1. we optimize the loss **L**
2. we adaptively set **λ**: if the KL is too high, we increase **λ**, if too low, we relax it

PPO, the one and the only

Let’s consider again what’s the problem... the source of the variance are the samples where we are going too far from the old policy. Can we do better than staying close to the reference?

PPO, the one and the only

Let's consider again what's the problem... the source of the variance are the samples where we are going too far from the old policy. Can we do better than staying close to the reference?

What about not considering them!

And indeed, it's what PPO will do, but let's see when to do it:

- if we have $\mathbf{A_t < 0}$, the next learning step, most likely the probability of taking that action will be decreased, thus the IS ratio will be less than 1, not causing any problem
- if we have $\mathbf{A_t > 0}$, the next learning step, most likely the probability of taking that action will be increased, thus the IS ratio will be greater than 1... here's the catch

PPO, the one and the only

Since we just saw that $\mathbf{A_t > 0}$ is the case were we might have problem, we just need to fix it, and indeed PPO does exactly that ($\mathbf{r_t}$ is just the IS ratio):

$$J(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \cdot \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \cdot \hat{A}_t \right) \right]$$

The reasoning above works out to be just the second term in the **min**, however we did not considered that even though we might have $\mathbf{A_t < 0}$, the probability at the second step of learning might actually have increased... in that case the min is used to avoid clipping that sample, and to still consider it.

Keep in mind that then the **min** term is the **clip**, if you actually clip the loss, **the gradient will be 0** (in other words, if you are too far from the old policy in that state, you stop learning from it)

Algorithm 1 PPO-Clip

Input: initial policy parameters θ_0, initial value function parameters ϕ_0

for $k = 0, 1, 2, \ldots$ **do**

Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.

Compute rewards-to-go $\hat{R}_t$.

Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.

Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.

Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.

end for

What about the good old DQN

For sure, DQN is still a great pick for many scenarios. Indeed, the reason policy gradients were introduced, was to tackle the problem of partial observability, but if we don't need this assumption, and we know that at some point we can use a deterministic policy, we can still use it, and it works like a charm.

Earlier you were introduced to the algorithm of DQN, with the following tricks:

- (Prioritized) Experience replay: helps with the IID problem
- target network: fix the target network used to calculate the target for Q-learning

However, other methods were proposed to improve it.

Double DQN

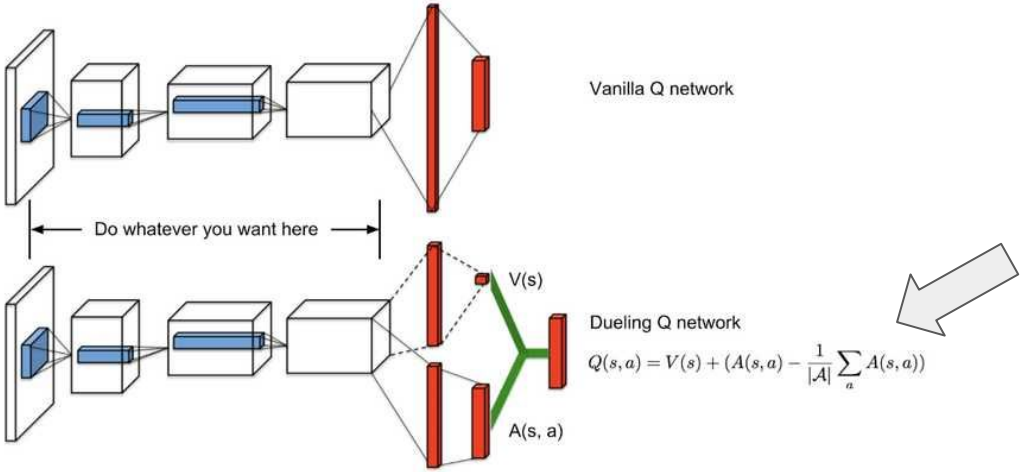
Q-learning has an intrinsic tendency to overestimate in function approximation, due to the **max** in the target calculation... DQN is definitely not immune to this problem, and overestimation in function approximators might even lead to divergence.

To address this problem, Double DQN used 2 Q networks instead of one, and uses one to find the target action, and another to estimate the Q of such action

$$Y_t^{\text{DoubleQ}} \equiv R_{t+1} + \gamma Q(S_{t+1}, \underset{a}{\text{argmax}}\, Q(S_{t+1}, a; \boldsymbol{\theta}_t); \boldsymbol{\theta}'_t)$$

Duelling DQN

Until now, we always tried to learn the Q values, however, as you have seen during the policy gradient lectures, Q values are nothing else then the Value plus the advantage **Q(s,a) = V(s)+Adv(v,a)**, and Duelling DQN does exactly that. It uses one head to estimate the value, and N others to estimate the advantages



This little additional part is necessary in order to force the V to estimate the actual value (in other words, that the average advantage is 0)

Other improvements:

Subsequently, other improvements have been proposed, even though the attention lately is shifting to other problems:

- Categorical DQN: instead of estimating the expected return for each action directly, it represents the distribution of possible returns (useful in safe RL)... aka as C51 because the used distribution is a categorical one with 51 bins
- Rainbow DQN: an improvement from DeepMind, that aims at aggregating all previous improvements (too many hyperparameter with no good guesses)
- Quantile DQN: tries to improve C51 using quantile regression (learning quantiles instead of expected values)

Now, a question for all of you:

Can we use this approaches in continuous action spaces?

Or even infinite discrete actions, for example for a Poisson policy

Can we use this approaches in continuous action spaces?

Well, theoretically, yes, practically, hardly not.

- policy-based: well now we have a problem... if earlier we could just have used a categorical distribution, now it's hard to have a concept of continuous distribution that can just model any distribution, so we need to rely on a family of distributions (gaussians), and that's not obvious at all (maybe we are in a continuous bounded action space)
- value-base: well, even if learning **Q(s,a)** might theoretically work, the problem is that **Q** is learnt under a policy, but now we cannot have a ϵ-greedy policy anymore, so DQN like methods cannot be easily applied

Next lecture we will see how to actually solve this issue, by using the nature of neural networks